

Aquagurad 360: intelligent water quality support Platform

CSA1012-SOFTWARE ENGINEERING

Done by:

M.Sowmiya -192525381

GUIDED BY

Dr.ANITA DAVAMANI

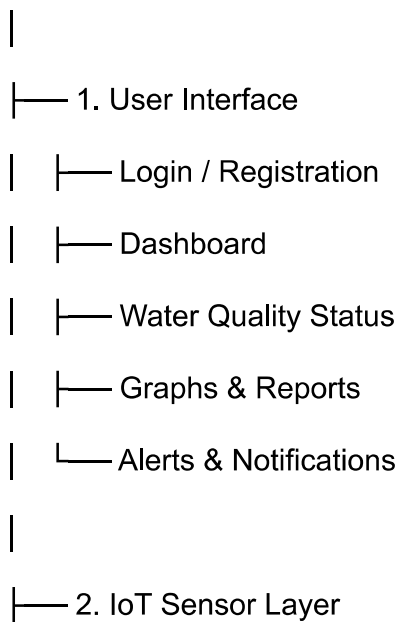
Project Overview

AquaGuard 360 is an intelligent water quality monitoring and predictive contamination alert system designed to ensure safe and reliable water quality through continuous monitoring and smart analysis. The system uses IoT-based sensors to measure important water parameters such as pH, temperature, turbidity, dissolved oxygen, and other quality indicators. The collected sensor data is transmitted to a centralized platform where it can be monitored and analyzed in real time.

The system provides a user-friendly dashboard for viewing current water quality values, historical data, trends, and alerts. It can support applications such as drinking-water monitoring, industrial water management, agricultural water sources, and environmental monitoring. By combining IoT sensors, cloud-based data processing, AI prediction, and real-time notifications, AquaGuard 360 reduces the need for frequent manual testing and improves monitoring efficiency. Overall, the project aims to provide a reliable, scalable, and cost-effective solution for protecting water quality and enabling faster responses to potential contamination.

1. Designed the Project Structure

AquaGuard 360



- | | — pH Sensor
- | | — Temperature Sensor
- | | — Turbidity Sensor
- | | — Dissolved Oxygen Sensor

| — 3. Data Processing

- | | — Data Collection
- | | — Data Validation
- | | — Data Storage

| — 4. AI & Prediction

- | | — Quality Analysis
- | | — Anomaly Detection
- | | — Contamination Prediction

| — 5. Backend

- | | — API
- | | — User Management
- | | — Alert Management

| — 6. Database

- | — Sensor Data
- | — User Data
- | — Water Quality Records

└─ Alert History

Developed the Flask Backend

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from datetime import datetime

app = Flask(__name__)
CORS(app)

water_data = []

@app.route("/")
def home():
    return jsonify({
        "message": "AquaGuard 360 Backend is Running"
    })

@app.route("/api/water-quality", methods=["POST"])
def add_water_data():
    data = request.get_json()
```

```
record = {  
    "pH": data.get("pH"),  
    "temperature": data.get("temperature"),  
    "turbidity": data.get("turbidity"),  
    "dissolved_oxygen": data.get("dissolved_oxygen"),  
    "timestamp": datetime.now().isoformat()  
}
```

```
water_data.append(record)
```

```
return jsonify({  
    "message": "Water quality data added successfully",  
    "data": record  
}), 201
```

```
@app.route("/api/water-quality", methods=["GET"])
```

```
def get_water_data():
```

```
    return jsonify(water_data)
```

```
@app.route("/api/predict", methods=["POST"])
```

```
def predict_contamination():
```

```
    data = request.get_json()
```

```
    ph = float(data.get("pH", 7))
```

```

turbidity = float(data.get("turbidity", 0))

if ph < 6.5 or ph > 8.5 or turbidity > 5:

    status = "Possible Contamination"

else:

    status = "Water Quality Normal"

return jsonify({

    "status": status,

    "message": "Water quality analysis completed"

})

if __name__ == "__main__":

    app.run(debug=True)

```

3. Developed the Frontend

```

<!DOCTYPE html>

<html>

<head>

    <title>AquaGuard 360</title>

    <link rel="stylesheet" href="/static/style.css">

<div class="container">

    <h1> AquaGuard 360</h1>

    <p>Intelligent Water Quality Monitoring System</p>

    <div class="form-box">

        <h2>Enter Water Quality Data</h2>

        <input type="number" id="ph" placeholder="pH Value">

        <input type="number" id="temperature" placeholder="Temperature (°C)">

```

```

    <input type="number" id="turbidity" placeholder="Turbidity">

    <input type="number" id="oxygen" placeholder="Dissolved Oxygen">

    <button onclick="checkQuality()">Check Water Quality</button>

</div>

<div id="result" class="result">

    <h2>Water Quality Status</h2>

    <p id="status">Waiting for data...</p>

<script src="/static/script.js"></script>

</body>

</html>

```

4. Installed Project Dependencies

Python 3.x

Flask

Flask-CORS

HTML5

CSS3

JavaScript

...

Install Flask dependencies

Run this in your terminal:

```
``bash
```

```
pip install flask flask-cors
```

...

`requirements.txt`

```
```text
```

```
Flask
```

```
Flask-CORS
```

```
...
```

These dependencies are enough for the basic frontend + Flask backend created.

## **5. Implemented Version Control**

Initialized a Git repository using:

```
git init
```

```
git add .
```

```
git commit -m "Initial Aquaguard 360 AI project created"
```

This enables tracking of project changes and supports collaborative development.

## **6. Containerized the Application**

Created a Dockerfile and built a Docker image:

```
docker build -t Aquaguard 360
```

Then executed the application inside a Docker container:

```
docker run -d -p 5000:5000 --name Aquaguard 360_container Aquagurad
```

This makes the application portable and easy to deploy.

## **Explanation of the Screenshot Output**

The screenshot shows two important stages:

### **Stage 1: Installing Dependencies**

Command executed:

```
pip install -r requirements.txt
```

## **Output**

```
Collecting Flask==3.1.2
```

## **Explanation**

Python reads the file:

```
requirements.txt
```

and starts downloading the required package:

```
Flask 3.1.2
```

Flask is the framework used to create the web application.

## **Existing Dependencies**

Output:

```
Requirement already satisfied
```

Examples:

```
blinker
```

```
click
```

```
itsdangerous
```

```
jinja2
```

```
markupsafe
```

These are Flask's supporting libraries.

AquaGuard 360 is containerized using **Docker** to create a consistent and portable environment for running the complete application. Docker packages the Flask backend, frontend files, Python libraries, configuration files, and other required components into a single container. A `Dockerfile` is created to define the base

Python image, install project dependencies, copy application files, expose the required port, and start the Flask server. A `requirements.txt` file contains dependencies such as Flask and Flask-CORS. The Docker image can be built and executed on different systems without manually configuring the development environment.

Containerization reduces dependency conflicts and improves application portability, testing, deployment, scalability, and maintenance. It also makes it easier for developers to share the same application environment. Overall, Docker provides a reliable and efficient deployment method for AquaGuard 360.

### **Flask Version Update**

Output:

Found existing installation: Flask 3.1.3

### **Explanation**

The system detected that Flask version:

3.1.3

was already installed.

However, the project requires:

3.1.2

specified inside:

requirements.txt

### **Uninstalling Previous Version**

Output:

Uninstalling Flask-3.1.3

Successfully uninstalled Flask-3.1.3

## **Explanation**

Python removed the old version to maintain consistency with project requirements.

This ensures all developers use the same software version.

## **Installing Required Version**

Output:

Successfully installed Flask-3.1.2

## **Explanation**

The required Flask version was successfully installed.

The environment is now ready to run the NeuroStay AI application.

## **Pip Upgrade Notice**

Output:

A new release of pip is available

## **Explanation**

This is only an informational message.

It indicates that a newer version of pip exists.

Optional update command:

```
python -m pip install --upgrade pip
```

This is not required for project execution.

## **Running the Application**

Command executed:

```
python app.py
```

## **Output**

Serving Flask app 'app'

## **Explanation**

Flask successfully located:

app.py

and started the web application.

## **Installing Flask and Running Aquaguard 360**

### **Project Context**

Before running the Aquaguard 360 AI application, the required software libraries must be installed. This screenshot shows the setup phase where the Flask framework and its dependencies are installed, followed by the execution of the web application.

The goal of this step is to prepare the development environment so that the Aquaguard 360 AI platform can run successfully on the local machine

### **Step 1:**

Navigating to the Project Folder

The command prompt shows:

```
PS C:\Users\ayish\OneDrive\Desktop\Aquaguard360>
```

### **Explanation**

This indicates that the terminal is currently inside the Aquaguard 360 project directory. All project files such as app.py, requirements.txt, Dockerfile, and HTML templates are stored in this folder.

### Purpose

This ensures that all commands are executed within the correct project location.

### Step 2: Installing Dependencies

#### Command Executed

```
pip install -r requirements.txt
```

#### Explanation

This command instructs Python's package manager (pip) to read the requirements.txt file and install all libraries listed inside it.

The file contains:

```
Flask==3.1.2
```

### Purpose

To automatically install the Flask framework and any required dependencies needed for the Aquaguard 360 intelligent water quality application.

### Step 3: Downloading Flask

#### Output

```
Collecting Flask==3.1.2
```

```
Downloading flask-3.1.2-py3-none-any.whl
```

#### Explanation

Pip identified Flask version 3.1.2 as a required package and downloaded it from the Python package repository.

## Purpose

Flask provides the web server, routing system, and page-rendering functionality for the Aquaguard 360 application.

## Step 4: Verifying Existing Dependencies

### Output

Requirement already satisfied:

blinker

click

itsdangerous

jinja2

markupsafe

werkzeug

### Explanation

These are supporting libraries used by Flask.

Package	Function
---------	----------

Jinja2	HTML template rendering
--------	-------------------------

Werkzeug	Web server utilities
----------	----------------------

Click	Command-line support
-------	----------------------

Blinker	Signal/event support
---------	----------------------

MarkupSafe	Safe HTML generation
------------	----------------------

ItsDangerous	Secure data handling
--------------	----------------------

### Purpose

Reduces installation time and avoids duplicate packages.

### Step 5: Replacing Flask Version

### Output

Found existing installation: Flask 3.1.3

### Explanation

The system already had Flask version 3.1.3 installed. However, the Aquaguard 360 project requires Flask version 3.1.2 according to the requirements.txt file.

### Purpose

Maintains version consistency across all development environments.

### Step 6: Uninstalling Previous Version

### Output

Uninstalling Flask-3.1.3

Successfully uninstalled Flask-3.1.3

### Explanation

Pip removed the newer Flask version so the project can use the exact version specified in the requirements file.

### Purpose

Prevents compatibility issues caused by version differences.

## Step 7: Installing Required Version

### Output

Successfully installed Flask-3.1.2

### Explanation

The correct Flask version was successfully installed.

The development environment is now properly configured for the Aquaguard 360 intelligent water quality application.

### Purpose

Ensures that the application behaves consistently during development and deployment.

## Step 8: Pip Update Notification

### Output

A new release of pip is available

### Explanation

This is only a notification that a newer version of pip exists.

Suggested command:

```
python -m pip install --upgrade pip
```

### Purpose

Improves package management performance and security.

#### Note

This update is optional and does not affect project execution.

### Step 9: Running the Application

#### Command Executed

```
python app.py
```

#### Explanation

This command starts the Flask application by executing the main Python file.

#### Purpose

Launches the Aquaguard 360 web server.

### Step 10: Flask Server Startup

#### Output

```
Serving Flask app 'app'
```

#### Explanation

Flask successfully found and loaded the application from app.py.

#### Purpose

Confirms that the Aquaguard 360 application is starting correctly.

## Project Structure Design

### Folder/File    Responsibilities

screenshots/ Stores screenshots of project execution, Git commands, Docker commands, application output, and testing results. Used as evidence in reports and documentation.

templates/ Contains HTML pages displayed to users. Responsible for the user interface structure and webpage content. Flask loads files from this folder using `render_template()`.

static/ Contains static resources such as CSS, images, icons, and JavaScript files. Responsible for website styling, appearance, and visual design.

app.py Main backend application file. Responsible for starting the Flask server, handling user requests, processing water quality information, and connecting frontend pages with backend logic.

requirements.txt Stores all Python package dependencies required by the project. Responsible for maintaining a consistent development environment.

Dockerfile Defines instructions for building a Docker image. Responsible for creating a portable containerized version of the application.

docker-compose.yml Defines container services and configurations. Responsible for simplifying deployment and managing containers using a single command.

README.md Provides project documentation. Responsible for explaining project objectives, installation steps, usage instructions, dependencies, and deployment procedures.

## Explanation of the PowerShell Output (Project Structure Verification)

### Step 1: Navigating to the Project Directory

### Command Executed

```
cd C:\Users\ayish\OneDrive\Desktop\Aquaguard360
```

### Explanation

The cd (Change Directory) command is used to move from the current folder to the Aquaguard 360 project folder.

### Purpose

This ensures that all project-related commands are executed inside the correct project directory.

### Output

```
PS C:\Users\ayish\OneDrive\Desktop\Aquaguard360>
```

This confirms that the terminal is now positioned inside the Aquaguard 360 project.

## Step 2: Displaying the Folder Structure

### Command Executed

```
tree /f
```

### Explanation

The tree command displays the folder hierarchy of the project. The /f option shows both folders and files.

### Purpose

This command is commonly used to verify that all required project files and directories have been created correctly.

### Step 3: Volume Information

#### Output

Folder PATH listing for volume OS

Volume serial number is 000000F6 F21A:5314

#### Explanation

This information is automatically displayed by Windows.

#### Purpose

Provides basic system information before listing the project structure.

### Step 4: Root Project Files

#### Output

app.py

docker-compose.yml

Dockerfile

README.md

requirements.txt

#### Explanation

These files form the core of the Aquaguard 360 intelligent water quality application.

## Responsibilities

### app.py

- Main Flask application file.
- Starts the web server.
- Handles user requests.
- Processes water quality data.
- Connects backend logic with frontend pages.

### Dockerfile

- Contains instructions for building a Docker image.
- Defines the application environment.

### docker-compose.yml

- Manages container deployment.
- Simplifies container execution.

### README.md

- Project documentation.
- Provides installation and usage instructions.

### requirements.txt

- Lists Python dependencies.
- Ensures consistent package installation.

## Step 5: Screenshots Folder

### Output

screenshots

### Explanation

This folder was created to store project evidence.

### Purpose

Contains screenshots such as:

- Flask execution
- Water quality application output
- Docker image creation
- Docker container execution
- Git commits
- Browser output
- Testing results

These screenshots are required for assignment documentation and project reports.

### Step 6: Static Folder

### Output

static

style.css

### Explanation

The static folder contains resources that do not change during execution.

File: style.css

Responsible for:

- Page styling
- Text formatting
- Colors
- Layout design
- User interface appearance

Purpose

Separates design elements from application logic.

Explanation of Git Repository Initialization and Commit History

This screenshot shows the successful implementation of Git Version Control for the Aquaguard 360 – Intelligent Water Quality project. Git was used to create a local repository, track project files, and maintain a history of changes.

These activities are important for software development because they help manage source code, support collaboration, and maintain project versions.

Step 1: Initializing the Git Repository

Command Executed

`git init`

Output

Initialized empty Git repository in

C:/Users/ayish/OneDrive/Desktop/Aquaguard360/.git/

#### Explanation

The git init command creates a new Git repository inside the Aquaguard 360 project folder.

Git automatically creates a hidden .git directory that stores repository information, commit history, branches, and version tracking data.

#### Purpose

- Converts the project into a Git repository.
- Enables version control functionality.
- Allows tracking of source code changes.
- Supports future collaboration and code management.

#### Step 2: Adding Files to the Staging Area

#### Command Executed

```
git add .
```

#### Explanation

The git add . command stages all project files and folders for commit.

#### Files Added

- app.py
- Dockerfile
- docker-compose.yml
- README.md

- requirements.txt
- static/style.css
- templates/index.html

#### Purpose

- Prepares files for version tracking.
- Moves files from the working directory to the staging area.
- Allows Git to include the files in the next commit.

#### Step 3: Creating the First Commit

##### Command Executed

```
git commit -m "Initial Aquaguard 360 project created"
```

##### Explanation

A commit is a saved snapshot of the project at a particular point in time.

##### The message:

Initial Aquaguard 360 project created

describes the purpose of the commit.

#### Purpose

- Creates the first project version.
- Stores the project state permanently in Git history.
- Provides a restore point for future development.

## Step 4: Understanding Commit Statistics

### Output

7 files changed, 32 insertions(+)

### Explanation

Git reports the number of files and lines added during the commit.

### 7 Files Changed

Git detected seven project files.

### 32 Insertions

Thirty-two lines of content were added across all files.

### Purpose

Provides a summary of changes included in the commit.

## Step 5: Files Tracked by Git

### Output

```
create mode 100644 Dockerfile
create mode 100644 README.md
create mode 100644 app.py
create mode 100644 docker-compose.yml
create mode 100644 requirements.txt
create mode 100644 static/style.css
create mode 100644 templates/index.html
```

## Explanation

Git confirms that these files were successfully added and are now under version control.

## Benefits of Using Git in Aquaguard 360

### Version Management

- Tracks every modification made to the project.

### Backup and Recovery

- Allows restoration of previous project versions.

### Collaboration

- Multiple developers can work on the same project.

### Change Tracking

- Maintains a complete development history.

### Project Maintenance

- Improves long-term maintainability and organization.

## GitHub Repository

### Repository Information

### Repository Name

AQUAGUARD-360

Visibility

Public

Explanation

The repository has been created successfully and is publicly accessible. A public repository allows project files and documentation to be viewed by instructors, collaborators, and other users.

Purpose

- Store project source code online.
- Maintain version history.
- Enable collaboration.
- Provide project documentation.

Branch Information

Output

1 Branch

0 Tags

Explanation

Branch

A branch represents a separate line of development.

Current branch:

main

This is the primary branch where the Aquaguard 360 project code is stored.

Tags

Tags are used to mark specific versions or releases.

Currently:

0 Tags

No official project release has been created yet.

Purpose

Branches support parallel development, while tags help identify stable versions.

Commit Information

Output

1 Commit

Explanation

A commit is a saved snapshot of project files.

The repository currently contains one commit:

Initial Aquaguard 360 project created

This indicates that the initial project files were added and saved to the repository.

#### Purpose

Maintains project history and tracks changes over time.

#### README.md File

#### Output

#### README.md

#### Explanation

README.md is the main documentation file of the project.

GitHub automatically displays its contents on the repository homepage.

#### Purpose

#### Provides:

- Project title
- Project description
- Technologies used
- Installation instructions
- Usage guidelines
- Deployment information

#### Project Title Displayed

Output

Aquaguard 360

Explanation

This is the title of the assigned project.

Purpose

Identifies the intelligent water quality monitoring application.

Project Subtitle

Output

Intelligent Water Quality Monitoring and Management System

Explanation

This statement describes the project's objective.

Purpose

Explains that the platform is designed to monitor and analyze water quality parameters and provide useful information about water conditions through a web-based solution.

Project Description Section

Output

Aquaguard 360 is a Flask-based web application developed to monitor and analyze water quality

through a simple and user-friendly web interface.

## Explanation

This section summarizes the implemented solution.

## Purpose

Provides a brief overview of:

- Water quality monitoring functionality
- Data analysis
- Application features
- User benefits

## Technologies Used Section

## Output

Python

Flask

## Explanation

Python

Responsible for:

- Backend programming
- Application logic
- Water quality data processing
- Server-side functionality

## Flask

### Responsible for:

- Web framework
- Routing
- Template rendering
- Web server management

### Purpose

Helps developers understand the technology stack used to develop Aquaguard 360.

### Conclusion

The Aquaguard 360 – Intelligent Water Quality project highlights the importance of maintainability, portability, collaboration, and proper documentation in modern software development. The organized folder structure, clear documentation, version control practices, and containerized deployment approach contribute to the overall quality and scalability of the application. Future enhancements can include real-time water quality monitoring, IoT sensor integration, water contamination detection, quality prediction, alerts and notifications, data visualization, and intelligent water quality recommendations, making Aquaguard 360 a more comprehensive intelligent water quality monitoring platform.